

WeKnora 质量评测基线与成本可观测 详细设计方案

徐博 · 湖南第一师范 · GitHub yushangcan

腾讯犀牛鸟开源人才培养计划 · 课题 3 · 2026 年 9 月 13 日

本方案解释在 WeKnora 官方系统上增加可复现评测、模型调用记录、费用解析、Embedding 结果缓存及 Wiki 提示词优化的完整设计。重点是让一次质量变化能够追溯到数据、配置、代码和用例，让模型消耗能够回查数据库，并用自动检查发现回归。

核心模块已有实现，标准 App 的合成端到端评测、数据库迁移和缓存专项检查已有 GitHub Actions 证据。真实模型账单、重建索引的缓存收益及 Wiki 厂商缓存收益仍待实测。本文的合成分数用于验证工程链路，不代表 CMRC2018 或生产模型成绩。

官方源码审计基线为 upstream/main 的 988cbb03。交付仓库为 <https://github.com/yushangcan/WeKnora>，开发分支为 `codex/evaluation-four-dimension-results`。冻结 Tag 为 `rhino-2026-final-3`；完整冻结 SHA 由仓库根目录 `submission.yaml` 记录。被 CI 编译执行的代码版本与包含文档的冻结版本分开记录。

1 需求范围与交付原则

课题三的题目为「质量评测基线与成本可观测」。需求来源是《WeKnora 开源实战课题》，材料格式来源是《WeKnora 腾讯犀牛鸟课题实战流程指引 2026》。下表以需求本身组织实现范围。

需求	实现方式	证据与边界
固定数据 模型 分块并持久化	数据文件指纹 配置快照 Run 与 Case 表	数据校验 回归测试 真实 App 落库及重启读取
一次运行输出质量 成本 耗时	复用质量指标 增加 Usage Cost Timing	合成报告有四类结果 未知费用保留 NULL
CI 及退化门禁	CLI compare 与 gate 自包含 Compose	正常与负向场景真实执行；分支保护需另配置
模型管理页面统计	调用事件表 汇总和明细 API Vue 面板	数据库查询及前端检查；真实 Provider 联调待补
相同输入复用 Embedding	按租户和模型配置隔离的 LRU 或 Redis	命中 去重 并发 失效 降级与 benchmark

需求	实现方式	证据与边界
Wiki 稳定前缀	固定规则和输出结构前置 动态内容后置	前缀和输出回归；真实命中率增益待补
八解析引擎横评	可选独立工作	按当前开发范围暂缓

本次保留官方业务框架和指标体系。新增逻辑尽量放在 `Evaluation`、模型装饰器和 `Repository` 中，避免在检索、重排、生成的每个分支重复统计。每个可验证改动单独提交 `Git`，便于按功能审阅和回退。

2 官方系统原本如何运行

2.1 入库与问答两条主线

官方后端使用 `Go` 和 `Gin`，前端使用 `Vue 3` 和 `TDesign`。`PostgreSQL` 保存标准版业务数据并支持检索，`Lite` 使用 `SQLite`；`Redis` 与 `asynq` 执行文档处理等异步任务。`Python DocReader` 处理部分文档解析。本次没有重写这些能力。

文档入库从上传文件、`URL` 或 `Markdown` 开始，先创建 `Knowledge`，再进入解析、分块、`Embedding`、向量和关键词索引。索引建立后，摘要、图谱或 `Wiki` 可继续处理同一知识底座。重复导入和重建索引会重复计算相同文本的向量，这决定了缓存应接在统一 `Embedding` 调用入口。

用户问答从会话历史和查询处理开始，经过向量或关键词召回、融合、可选 `Rerank`、上下文组装，再调用 `Chat` 或 `ChatStream` 并输出引用。评测调用现有 `KnowledgeQAByEvent`，因此可以观察实际流水线返回的段落顺序和最终答案。

2.2 原有评测与具体缺口

官方已经实现 `Precision`、`Recall`、`NDCG@3`、`NDCG@10`、`MRR`、`MAP`，以及 `BLEU` 和 `ROUGE` 的六项答案指标。原始入口通过 `POST` 创建任务、`GET` 轮询，数据来自五个 `Parquet` 文件。评测能力存在，但状态主要在进程内存中，缺少长期保存的运行身份、配置、用例证据和跨版本质量门禁。

只保存总分无法解释退化。例如果某个问题检索出了正确 `Chunk`，但没有还原数据集 `PID`，`Recall` 会被错误计算；如果入库尚未完成便开始问答，分数也会偏低。设计必须先让指标输入可信，再增加报告和 `CI`。

成本侧的缺口同样具体：日志里的调用用量无法可靠按模型和时间聚合，厂商返回的缓存命中不能代表应用已有结果缓存，`Token` 数也不能直接当作货币金额。本次为三者建立独立口径。

3 总体架构与职责分配

整体流程为：固定数据及配置 → 创建 Evaluation Run → 执行真实 RAG → 收集质量和调用事实 → 保存 Run 与 Case → 查询历史和比较 → CLI 门禁 → 保存验收报告。成本流程为：模型调用 → Provider 用量适配 → 统一装饰器 → 事件落库 → 按租户和时间聚合 → 模型页面展示。

模块	主要职责	代码入口
Evaluation Service	编排临时知识库 用例执行 清理和状态	internal/application/service/evaluation.go
Dataset Service	五文件校验 排序 数据指纹	internal/application/service/dataset.go
Evaluation 核心	配置快照 上下文 Observer Collector	internal/evaluation/
Evaluation Repository	Run Case 原子进度 历史查询 租约	internal/application/repository/evaluation*.go
Model Usage	装饰器 Provider 用量 成本解析	internal/models/usage/
Embedding Cache	缓存键 LRU Redis 去重与协调	internal/models/embedding/result_cache.go
Wiki Prompt	固定前缀及模板输出约束	internal/agent/prompts_wiki.go
CLI 与 CI	启动 比较 门禁 合成验证	cmd/evaluation/ 与 ci/evaluation/
前端	评测历史 比较 模型调用统计	frontend/src/views/evaluation/ 与 settings/ModelUsagePanel.vue

以 Embedding 为例，从调用方向看是业务 → 结果缓存 → 用量装饰器 → 原 Provider 能力。缓存命中在最外层返回，不会生成虚假的 Provider 调用事件；未命中才进入用量记录。现有并发、批处理和追踪逻辑保持在其既有层次，避免重复包装造成重复计数。

评测 Observer 保存本次 Run 的事实，全局 ModelUsage Recorder 保存可跨运行查询的调用事件。两者可以通过 run_id 和 case_id 关联，汇总时不能把同一次调用相加两遍。

4 评测数据与可复现配置

4.1 五文件数据契约

queries.parquet 保存问题，corpus.parquet 保存检索语料，qrels.parquet 关联问题与相关段落，qas.parquet 关联问题与参考答案，answers.parquet 保存答案文本。加载器检查 ID、重复记录、关系引用与有效评测用例，按稳定顺序读取。

每个文件记录文件名、字节大小、SHA-256；整体 fingerprint 按规定文件顺序包含名称与原始字节。只比较 dataset_id 无法发现同名数据已被替换，因此历史比较采用内容指纹。缺文件、重复关系或引用缺失必须失败，不能静默改变评测样本数。

评测建库使用完整 corpus，包含与当前问题无关的干扰段落。如果仅灌入正确答案所在段落，检索难度会被人为降低。本次灌入完成后等待可检索状态，再启动 Case。

4.2 PID 贯穿实际检索

数据集 PID 与业务 Chunk ID 不同。入库时在 metadata 写入 evaluation_pid，一个段落被分为多个 Chunk 时共享其数据集身份。Observer 记录 Search、Rerank 和最终指标输入的 PID，按规则去重。

优先使用最终重排结果，未执行重排或 API 错误回退时使用召回结果。流水线以 RerankCompleted 区分“重排完成但结果为空”和“未完成重排”：前一种必须按空结果计算，不能把原始召回列表拿回来补分。无法映射的返回项保留负数占位并累计 unmapped_result_count，不能从列表中直接删除而抬高 Precision。Case 同时保存 ground_truth_pids，便于逐项解释漏召回、顺序变化和身份映射错误。

该问题由新增召回反向 CI 实际发现：相关结果已被重排过滤，旧指标仍退回 SearchResult，造成 Recall 虚高。修复提交 8945b5b2 让指标和 Case 证据采用同一选择条件，并将指标版本升为 retrieval-generation/v3；旧 v2 和新 v3 历史结果不应直接比较。指标公式本身保持不变，修正的是指标输入来源。

4.3 CMRC2018 的固定来源

转换器位于 dataset/cmrc2018/convert/。固定数据 revision 为 137f2c45a24275fb68f6961c4d357f46288886aa，validation 文件 SHA-256 为 c3d3116e5845b8d3c44a863c1e2f973817c9f10a52e503b079ac49d4366f78b2。转换前读取一次源字节并验哈希，再解析同一份字节，防止输入和声明来源不一致。

本机对固定来源的实际转换检查得到 3219 行问题、848 个不同上下文；3216 行至少有一个有效抽取答案。审计出 210 条无效答案标注和 3 行无有效答案。默认稳定选取 200 个有效问题，仍保留全部 848 个上下文作为检索语料。中文答案偏移按字符边界核验，不将字符偏移直接当作 UTF-8 字节偏移。

这些是数据审计结果，尚未用真实模型运行完整 200 问对照。原始及生成数据不随 Git 提交，下载、哈希验证、转换方式见 dataset/cmrc2018/README.md；五项离线测试在没有原始文件时也能执行，完整官方数据测试则要求原始文件存在。

4.4 配置快照与可比性

evaluation-config/v2 记录数据指纹，Embedding、Chat、Rerank 身份，分块大小和 overlap，检索 Top-K 和阈值，生成参数、Prompt 指纹，以及代码 SHA、构建状态、指标版本和结果版本。规范化内容生成 config_hash。

密钥、完整 Endpoint、Prompt 正文和答案正文不进入新增观测快照；涉及内容的字段保存指纹。模型配置变化、语料变化或指标版本变化必须可以追溯。供应商线上模型和服务端参数仍可能变化，因此配置可追溯不等于真实模型永远确定性。完整合成 CI 使用固定响应解决工程测试的外部随机性。

5 单次运行四维结果与持久化

5.1 运行生命周期

Run 创建时即落库为 pending，初始化后进入 running。Case 完成时保存当前结果与进度，最终根据完成情况收敛到 success、partial 或 failed。全量成功与部分有效结果必须区分，任务出错不能仅把失败文案塞进一个成功状态。

执行过程中临时 KnowledgeBase 与 Knowledge 由评测负责清理。正常结束、异常、取消和持久化失败都进入清理路径；清理失败作为 Warning 保留。清理时间与评测时间分开记录，使总耗时可以解释。

5.2 数据库结构

表	核心字段	写入与读取约束
evaluation_runs	run_id tenant_id status config_snapshot result_snapshot revision 生命周期及租约	创建先落库；进度和终态校验归属；按租户查询
evaluation_run_cases	run_id case_id 状态 质量 Usage Timing Evidence	按 Run 与 Case 身份更新；历史分页返回
model_usage_events	call_id tenant_id 模型快照 operation source Tokens Cache Cost	调用完成记录；按模型 时间和来源聚合

Run 快照不重复内嵌全部 Case；读取时再组合，降低进度更新时复制所有 Case 的开销。Repository 的原子保存保持 Run 进度与 Case 结果一致，revision 用于防止旧快照覆盖新状态。PostgreSQL 本次迁移链到 000095，SQLite 到 000017，具体冻结内容以 migration 文件为准。

5.3 四类结果的解释

检索准确性复用原指标实现：Recall 反映相关段落是否找回，Precision 反映返回项相关比例，MRR、MAP、NDCG 反映排序质量。报告必须同时保留指标输入 PID；分数偏低时先检查对应 Case，再决定是否修改检索策略。

答案质量复用 BLEU-1、BLEU-2、BLEU-4、ROUGE-1、ROUGE-2、ROUGE-L。它们主要衡量词面相似度，不能直接解释为事实正确率、引用忠实度或幻觉率。本方案没有把新增 Judge 的主观分数混入现有指标。

成本结果与 Usage 分开：调用次数、Token、缓存读取属于用量；amount、currency 和来源才表示金额。没有金额依据的 Run 仍可成功完成，其 cost.amount 为 NULL。耗时结果包含准备、评测、清理、总墙钟时间和 Case 分布；并发模型调用耗时之和可能大于墙钟时间，二者不能混用。

5.4 重启与多实例租约

完成后的历史读取全部来自数据库，因此 App 重启不应丢失已完成 Run。运行中崩溃则需要防止其他实例误关闭活跃任务：每次执行分配 owner_id，初始租约两分钟，每 30 秒续租，并在进度及终态写入时检查 owner 和有效期。

恢复扫描仅处理无租约或已过期的未终态任务，更新前再次检查 revision 和租约。失去归属的执行取消上下文，避免继续写入；有进度的中断任务保留 partial，没有进度则 failed。owner_id 不暴露到公共 JSON。

当前恢复是中断状态收敛，不会自动接管并重跑剩余 Case。完成后重启读回已有端到端证据，双 App 运行中故障演练仍待补。升级和回滚前应停止旧评测任务，避免旧进程绕过新租约约束。

6 模型调用与费用可观测

6.1 统一调用事件

Chat、Stream、Embedding、Rerank 和多模态适配路径统一生成 ModelUsageEvent。事件记录模型名称快照、租户、操作、来源、开始结束时间、成功状态和可观测用量；评测上下文额外带 run_id、case_id，Wiki 等业务来源也可筛选。

Stream 需要在结束、取消或异常时闭合事件。批量输入保留 item_count，并避免将批量协调层与下层相同请求重复记录。调用事件表示装饰器观察到的逻辑调用；未显式接入的 SDK 内部重试不应宣称逐网络请求完整覆盖。

记录器使用数据库 Repository，写入失败不会改变原模型的返回结果。这个可用性取舍意味着数据库故障时存在观测缺口；当前并没有持久消息队列补偿每条事件。因此统计页是已记录调用的事实视图，不能冒充厂商账单总账。

6.2 Provider 用量适配

部分 Embedding 或 Rerank 接口只返回向量或排序，直接改变所有公共接口成本较高。实现使用调用作用域的 ProviderUsage sink，让适配器读取响应中的用量、金额或 request ID，装饰器在调用结束时提取这些证据。

已扩展的路径包括部分 Aliyun、Volcengine Embedding，Aliyun、Jina、Zhipu Rerank，以及 OpenAI-compatible VLM 和 ASR 调用事实。不同 Provider 的字段覆盖不同；有调用事实不代表一定有 Token、cache 或费用数据。

6.3 未上报与零值

情况	保存与展示
明确上报 Token 或金额为零	保存 0
没有上报 Token	保留 NULL 同时保留上报调用计数
没有缓存字段	cache unreported 命中率不可计算
费用缺少金额 币种或计价依据	amount 为 NULL unavailable 或 partial
多币种汇总	不跨币种直接累加

最近修复的缓存报告问题是：CacheReported 为 false 时曾跳过整条事件，连已明确上报的 Prompt Token 也丢失。修复后 Prompt Token 独立累计；缓存 Token 比例仅选同时具备 Prompt 与 Cache Read 字段的同一批调用，避免拼接不同样本的分子和分母。

6.4 费用解析及当前限制

PricingResolver 优先保存 Provider 明确金额；没有金额时，只有 ProviderUsage 具备币种、billable units 且命中版本化目录，才计算 catalog_calculated。目录通过 WEKNORA_PRICING_CATALOG JSON 数组配置，使用 provider、model、unit、currency、price_minor_per_unit 和 pricing_version。

UsageSource 表示用量来源，CostSource 表示金额来源，两者分开保存。例如用量来自 provider，金额可能来自 provider_reported，也可能来自 catalog_calculated。不能用同一字段覆盖二者。

内部采用整数货币最小单位并做精确转换，缺价、币种未知、不完整单位或无法精确表达时保留 NULL。当前最小单位和整数量价模型对细粒度亚分 Token 单价有限制，不能直接填一个虚构单价来补齐费用。自动获取价格、历史价格变更管理、Provider 账单异步对账和真实账单核验均未完成。

6.5 API 与页面

模型汇总入口是 GET /api/v1/models/usage/summary，明细入口是 GET /api/v1/models/usage/events。后端从认证上下文取租户，支持模型与时间等筛选；前端 ModelUsagePanel 展示调用、成功失败、Token 上报数、缓存、耗时、费用及来源，并能展开分页事件。

评测页面提供历史 Run、Case 证据和比较结果。前端选择逻辑保留跨页已选 Run，比较 Usage 与质量分别做兼容性判断。类型检查、构建和评测页面专项测试已通过；本次自包含 CI 不启动浏览器，模型统计页面的真实 Provider 可视验收仍需补录。

7 Embedding 结果缓存

7.1 缓存身份与开关

缓存默认关闭，通过 `WEKNORA_EMBEDDING_CACHE_ENABLED=true` 开启。有 Redis 时选 Redis，否则使用有界进程内 LRU；默认 TTL 为 168 小时，内存容量为 2000 条，可通过配置调整。

缓存 key 组合版本前缀、租户哈希、模型指纹和原始文本 SHA-256。模型指纹包含模型身份、Provider、BaseURL、更新时间、维度、截断参数和额外配置，API Key 与自定义请求头不进入 key。文本不做隐式 trim 或 Unicode 归一化，避免改变向量输入语义。

模型配置更新使指纹变化，旧项自然失效并等待 TTL 清理。不同租户不共享命中。同一 key 返回向量副本，避免调用者修改结果后污染缓存。

7.2 批量与并发行为

请求先查缓存，命中直接返回；未命中按 key 去重和协调，只把真正需要计算的输入交给 Provider。批量响应要核对数量、顺序、维度及有限浮点数，再写入缓存并恢复原输入顺序。

进程内通过 `singleflight` 合并同时到来的相同 key。Redis 多实例使用带 owner token 的分布式锁，持有者计算写入，等待者有界退避读取；等待超时可直接走 Provider，优先维持业务可用。续租和释放必须核对 token，避免旧请求释放新持有者的锁。

缓存读写故障采用 `fail open`，即回退到原 Provider。空向量、错误维度、NaN 或 Inf 不缓存。故障时仍可能增加 Provider 调用，这属于退化策略，不应算作缓存正确命中。

7.3 统计与对照实验

`hit_items` 和 `miss_items` 按原始输入位置计数，`provider_items` 按去重后发送的文本数计数。例如同文本出现两次且都未命中，Miss 是两项，Provider Item 可以是一项。Get Error 按实际缓存读操作计数，`singleflight` 内部复查失败也会增加一次。

已有 Linux benchmark 对 disabled、cold、warm 三种模式作函数级测试，使用 Fake Provider。此实验可验证调用抑制和包装开销；它没有网络、真实分词和模型服务延迟，不能据此宣布重建索引提速或费用下降。

真实验收应在相同语料、模型版本、维度、并发和机器上执行三组重建索引，记录 Provider requests、items、应用缓存命中、总耗时、错误和 Token。每组保留原始调用证据，用 disabled 与 warm 的请求数计算减少比例；目前该实测未执行。

8 Wiki 提示词前缀优化

模型厂商的前缀缓存依赖连续、稳定的输入字节。Wiki 多个批次重复使用规则、输出 Schema 和共享上下文，适合将固定内容前置，把本轮 chunk、页面正文和批变量移到后面。

实现保持原有模板占位符和输出约束，逐个模板调整顺序，并用前缀测试核对稳定段落不随动态输入变化。输出回归 fixture 检查字段和引用结构，避免为缓存破坏 Wiki 生成契约。

应用结果缓存与 Provider 前缀缓存作用不同：前者直接跳过 Embedding 请求，后者仍发起模型调用，只可能减少厂商重复处理的输入。页面中的厂商 cache hit 不能拿来替代应用 Embedding 命中率。

真实对照应固定一份 Wiki 语料、模型、生成设置和模板，以旧顺序为 baseline、新顺序为 candidate，多次记录 Provider 明确返回的缓存读取 Token、总 Prompt Token、命中调用比例、耗时及输出质量。当前只有稳定前缀与合成字段校验，没有真实命中率提升数字。

9 历史比较与质量门禁

9.1 比较规则

比较 API 从数据库读取 baseline 和 candidate。质量比较的强制条件是数据内容指纹、指标版本、结果版本、终态及质量字段齐全；Usage 比较独立判断可比性。不满足条件时返回原因，不生成可用于放行的质量差值。

模型、分块和生成参数保存在快照中供对照，当前一般比较接口不强制两次 config_hash 相等，因为它也需要支持更换模型或参数后的实验。调用者必须控制实验变量；自包含 CI 对两次正常重复运行额外要求 config_hash 相同。不能把一般比较接口描述为对所有配置差异都自动拒绝。

CLI 提供默认启动评测、compare 和 gate 三种操作。gate 需要真实存在且成功的 baseline、至少一个候选、唯一 Run 身份、可比较状态和有效指标差值。缺 baseline、重复候选、失败任务、缺指标和不兼容比较都返回非零。

门禁按指定指标的绝对差值检查：candidate 比 baseline 下降超过允许容差便失败，并打印 Run、指标名、差值和阈值。合成场景使用零容差，避免小用例的明显变化被放过；真实模型基线应根据重复试验波动预先确定阈值。

9.2 自包含验收环境

统一入口是 bash scripts/evaluation_ci.sh。脚本用独立 Compose project 构建当前 checkout 的标准 App 和 CLI，启动 ParadeDB/PostgreSQL、Redis 和合成 Provider；注册临时用户、签发评测专用 Key，再通过真实 HTTP API 执行评测。

构建使用 Go 1.26、Linux 和 CGO，安装标准服务编译需要的 SQLite 头文件。运行服务在内部网络，不绑定宿主端口，不读取开发者生产 .env。首次拉镜像、依赖和构建需要网络，因此一条命令不等于首次完全离线。

固定 fixture 是 France/Paris 与 Japan/Tokyo 两个问题，加 Mars 干扰段落；8 维合成向量和确定性 Chat/Rerank 响应。所有质量计算、数据加载、认证、迁移、RAG 与持久化都来自标准 App，只把外部模型替换为 Stub。

场景	注入方式	预期证据
baseline	正常响应	两个用例 Recall MRR MAP 为 1 答案指纹正确
candidate	同配置重复运行	config_hash 相同 比较和门禁通过
degraded	模型回答 Incorrect	答案指标下降 gate 拒绝
retrieval degraded	相关重排分数降为 0.01	实际相关段落被过滤 Recall 为 0 gate 指出 recall
restart	完成后重启 App	四次 Run 八个 Case 从数据库读回 结果一致

负向场景通过真实 Provider HTTP 接口改变响应，不手工修改报告。召回测试检查最终指标输入确实没有相关 PID，再检查 Recall 为零及门禁错误名。正常场景另有绝对质量下限，避免 baseline 和 candidate 同时坏掉却因差值为零而通过。

当重排过滤为空时，App 使用无上下文回退而不调用 Chat。这条分支仍须保存实际模型调用事实，但当前 generic Embedding 和 Rerank 适配路径未必上报 Token；验收因此保留未上报状态，不为满足断言虚构一次 Chat 调用。金额仍须为 NULL。

9.3 CI 的身份与证据

Go 构建保留 vcs.revision 和 vcs.modified，运行报告的 commit_sha 必须等于 checkout。数据 manifest 与实际挂载的五个文件逐字节哈希核对。数据库断言检查完成数、Case 数和终态租约释放，最后导出专用评测表和报告。

线上收尾修复了三个环境问题：全量 App 构建缺少 sqlite3.h，PostgreSQL 初始化临时 Unix socket 导致健康检查过早通过，以及容器以 root 写入 0600 报告导致宿主 uploader 无法读取。分别通过安装头文件、改用 TCP 健康检查、对齐验收容器 UID/GID 解决。

专项 workflow 还显式使用 Bash 和 pipefail，避免 `go test | tee` 把测试失败掩盖；migration version 的 stderr 也被捕获并检查非空。Actions 原 artifact 保留 14 天，提交目录保留精选原始报告与来源哈希，便于过期后继续审阅。

9.4 默认分支与强制门禁的边界

workflow 已声明 push、pull_request、workflow_dispatch 和每周定时入口，优化分支的 push 已实际触发。GitHub schedule 只从默认分支执行，当前成果尚在优化分支，不能声称该版本已在默认分支自动定时运行。

本次查询显示用户仓库 main 未开启分支保护。workflow 失败会显示检查失败；若要让仓库强制禁止合并，维护者还需将相应检查设为 required，并正确处理路径过滤的跳过场景。该默认分支部署与保护配置未在本次提交中改动。

10 实际验收结果及证据层次

最终自包含 CI 是 Actions 34759714335，实际编译代码为 f0a96a013269e35c2b265181fa5c88af8f3fcdf2。已下载的报告确认四次 Run、八个 Case、正常 gate 通过、答案和召回负向 gate 均拒绝；重启后读回一致，数据库终态租约释放。正常 Recall 为 1，召回退化 Recall 为 0，日志明确写出 recall delta -1.000000。这是 v3 口径的实际合成证据。

迁移专项 Actions 34759294412 在代码 8945b5b2 上实际执行全量升级、重复升级、回滚和恢复，并通过空重排结果回归。首次与重复版本为 95，回滚为 94，之后恢复；Run、Case、Model Usage 三表以及费用来源和租约字段通过检查。缓存专项 Actions 34756888995 在 5cad1109 上成功执行 Embedding 包测试和三模式 benchmark。

提交目录归档 30 份精选原始证据，来源、被测试 SHA、文件大小和 SHA-256 见 evidence/验收索引.json。gate 日志仅改扩展名为 txt，内容未改。版本冻结包含文档更新，因此最终 Tag 对应的 SHA 与上述编译测试 SHA 分开标识。

本地已有 Evaluation CLI、CMRC 转换器测试、前端类型检查和生产构建证据，评测选择与比较共六项测试通过；新增负向场景后 Python harness 为 12 项测试。Windows 原生部分 Go 测试受既有 pg_query CGO 环境限制，相关包通过 Linux CI 验证。本机 Docker Desktop 无法启动，因此完整容器链路证据来自 GitHub Actions。

本次不报告真实模型提升百分比。合成 Recall 为 1 或 0，表示测试夹具条件被满足；金额 NULL 表示无费用依据；Fake Provider benchmark 表示函数级调用减少。它们分别证明软件行为，不能互相替代。

11 运行与后续验收方案

11.1 干净环境复现

准备 Git、Bash、Docker Engine 与支持 up --wait 的 Docker Compose v2。推荐使用 Linux；Windows 使用 Docker Desktop 的 Linux 容器模式及 Git Bash。克隆并检出冻结 Tag 后运行：

```
git clone https://github.com/yushangcan/WeKnora.git
cd WeKnora
git checkout rhino-2026-final-3
bash scripts/evaluation_ci.sh
```

上述最后一条命令完成构建、迁移、评测、比较、重启验证和清理。成功退出 0；报告在 tmp/evaluation-ci/ 下带时间与进程号的目录中。质量常量与契约应一致，运行 ID、时间戳、耗时和构建 SHA 随实际执行变化。

11.2 接入真实数据与 Provider

先根据 CMRC2018 README 下载并验哈希，转换为固定 200 问与完整语料。单独部署可用 App，配置同一知识库的分块和 Embedding、Chat、Rerank 模型；将真实凭据放环境或安全配置中，不写进报告。

Evaluation CLI 从 WEKNORA_BASE_URL、WEKNORA_API_KEY、WEKNORA_TENANT_ID 读取服务信息，从 EVALUATION_DATASET_ID、EVALUATION_KNOWLEDGE_BASE_ID、EVALUATION_CHAT_MODEL_ID、EVALUATION_RERANK_MODEL_ID 读取选择。执行 `go run ./cmd/evaluation` 保存报告，记录返回 Run ID；第二次运行后用 baseline 和 candidate IDs 执行 `compare` 与 `gate`。

真实对照至少固定数据、模型版本、分块、检索、生成参数和并发，预先选择评价指标与容差，分别报告每个 Case、总体质量、调用、可观测费用和耗时。出现质量下降先定位 Case 证据，不能只把阈值放宽使门禁通过。

11.3 剩余项与完成标准

工作	下一步动作	可认定完成的证据
真实费用和页面联调	调用已支持模型 核对事件 API 与页面	Provider 返回与数据库字段对应 未上报明确保留
Embedding 实测	同语料 disabled cold warm 重建索引	实际请求数 延迟 命中与错误的原始记录
Wiki 实测	固定语料 旧新前缀及重复调用	厂商 cache 字段 输出质量和耗时对照
默认分支门禁启用	维护者合入 workflow 并设 required checks	默认分支定时运行及被拒绝的回归 PR
多实例故障	双 App 执行并终止 owner	其他活跃 Run 不受影响 过期任务收敛 无旧 owner 写入

后续另行讨论断点续跑、账单对账和亚分计价扩展。八解析引擎横评为选做，不计入当前基础开发范围。未完成项不会因本次冻结而被隐藏，下一次实测应继续生成新的版本和报告。

12 分阶段提交与阅读顺序

阶段	关键提交	阅读重点
评测可信与持久化	9e85643e 8ebcd4c4 0031eca9	终态 租户隔离 部分完成和持久化重试
历史比较与前端	13c99be0 1a0ad70a 49c8f9e8	历史查询 比较契约 跨页选择
调用与模型页面	65fccbdd 15ab50cb d6f0d517 73093bb9	事件模型 Repository Recorder API 页面
用量和费用扩展	7b4fa365 34b97e93 1fb5fced 3843480c	Provider sink 金额来源 价格目录 多模态
结果缓存与 Wiki	2088c2fe 7bb5146e d0674520 0f034129	缓存身份 分布式协调 前缀稳定性
CLI 与 CI	d299ce97 ff9b2fce cd78022b e5c23a2c	比较门禁 迁移 自包含环境
多实例保护	a4bab041 3ec3f1a9	owner lease heartbeat 写入归属
数据与线上收尾	b7ad630f f4cf0e6b 6e809297 16e9ced1	CMRC 来源 统计修正 构建与 artifact
召回负向门禁	1f114dda	真实过滤导致 Recall 下降及错误名验证
空重排结果修正	8945b5b2	区分重排完成与跳过 修复 Recall 虚高 升级指标版本

建议先看入库和问答流程，再读 Evaluation Service 与 Repository，然后沿模型装饰器进入 Usage 和缓存，最后运行 CLI 对照报告。已有历史设计文档 docs/WeKnora官方主线到优化完成详细设计方案-v1.md 保留阶段快照；本文和提交 README 描述当前交付状态。

本次贡献是在官方已有功能上补足工程流程，并非重新实现官方 RAG、全部评价指标或模型 SDK。分支中包含 upstream 同步提交，审阅贡献时应以官方审计基线和上述功能提交定位，不能把整个 fork 与旧 main 的差异都计为个人新增。